

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO N.º 2

Laboratorio: Concurrencia e IA como motor primario

Unidad 1 · Integridad, Transacciones y Concurrencia - Semana 2

Campo	Dato
Materia	Base de Datos II
Alumno	Nicolás Moron
Proyecto integrador	Food Store - sistema de gestión de pedidos de un negocio de comidas
Motor	PostgreSQL
Fecha	10/09/2026

Nota de alcance. La resolución utiliza el esquema Food Store del TP1. Los comandos de concurrencia están preparados para reproducirse en dos sesiones PostgreSQL. Como la evidencia exigida por la cátedra debe provenir del motor real, las salidas numéricas incluidas se presentan como referencia reproducible y deben confirmarse en la base local antes de la entrega definitiva.

0. Esquema de trabajo utilizado

Se toma como base el esquema relacional Food Store construido en la Semana 1: categorías, productos, clientes, pedidos y detalle de pedidos. Para los ejercicios de concurrencia se usan principalmente producto y pedido_detalle, porque permiten observar cambios de stock, inserciones y bloqueos sobre filas concretas.

Tabla	Atributos relevantes para este TP
categoria	id, nombre, activo
producto	id, nombre, precio, stock, categoria_id, activo
cliente	id, nombre, email
pedido	id, cliente_id, fecha, forma_pago
pedido_detalle	pedido_id, producto_id, cantidad, precio_unitario

1. Parte 0 - Puesta a punto y protocolo de seguridad

1.1. Flujo con IA y repositorio

El flujo propuesto sigue el criterio de la guía: la IA puede producir una propuesta, pero la decisión se conserva del lado humano. Antes de cualquier modificación se trabaja en modo Plan, se revisa el diff y recién después se permite la escritura sobre el repositorio.

```
cd food-store-db/
opencode
/init
# revisar y commitear AGENTS.md
git add AGENTS.md
git commit -m "docs: documentar contexto del esquema para agentes IA"
```

Para Kiro, se mantienen steering docs con las convenciones del esquema: nombres en singular, claves BIGINT identity, borrado lógico mediante activo, tipos ENUM y política de integridad referencial.

1.2. protocolo_seguridad.md

```
# Protocolo de seguridad - Food Store

## 1. Copia
createdb food_store_tp2
pg_dump -Fc food_store_dev > backups/food_store_dev_pre_tp2.dump
pg_restore -d food_store_tp2 backups/food_store_dev_pre_tp2.dump

## 2. Transacción
BEGIN;
-- aplicar primero el cambio sobre food_store_tp2
-- ejecutar pruebas válidas e inválidas
ROLLBACK;

-- solo después de revisar el efecto:
BEGIN;
-- aplicar cambio validado
COMMIT;

## 3. Respaldo antes de DDL
pg_dump -Fc food_store_tp2 > backups/food_store_tp2_antes_ddl.dump

# Regla adicional
# Verificar siempre la conexión actual antes de escribir:
SELECT current_database(), current_user;
```

Este archivo debe commitearse antes de continuar. La verificación de `current_database()` evita una de las fallas más graves descritas por la consigna: ejecutar una operación técnicamente correcta sobre el entorno equivocado.

```
git add protocolo_seguridad.md
git commit -m "docs: agregar protocolo de copia transaccion y respaldo"
```

2. Parte 1 - Integridad versionada con OpenCode

2.1. Reglas elegidas y spec

Se seleccionan reglas que complementan las restricciones declarativas del TP1 y que conviene garantizar también en el motor, especialmente cuando existan varios clientes o procesos escribiendo en paralelo.

Regla	Spec exacta
R-A	No permitir insertar ni reactivar un producto activo si su categoría está inactiva. Tabla producto; columnas activo y categoria_id; tabla categoria; columna activo.
R-B	No permitir agregar un pedido_detalle que referencie un producto dado de baja. Tabla pedido_detalle.producto_id contra producto.activo.
R-C	No permitir cantidad ≤ 0 ni precio_unitario < 0 en pedido_detalle. Si estas CHECK ya existen, OpenCode debe detectarlo y no duplicarlas.

2.2. Prompt utilizado en modo Plan

Revisá schema.sql del proyecto Food Store sin modificar archivos todavía.

Quiero garantizar en PostgreSQL estas reglas:

- 1) producto.activo no puede quedar TRUE si la categoria referenciada está inactiva;
- 2) pedido_detalle no puede insertar líneas para un producto inactivo;
- 3) pedido_detalle.cantidad debe ser > 0 y precio_unitario ≥ 0 .

Primero indicá qué restricciones ya existen y evitá duplicarlas. Para las reglas que requieren consultar otra tabla, proponé funciones trigger y triggers.

No uses DELETE físico. Mostrá plan, archivos a tocar y pruebas SQL válidas e inválidas.

2.3. Script de restricciones propuesto

```
-- 1) Producto activo exige categoria activa
CREATE OR REPLACE FUNCTION fn_producto_categoria_activa()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.activo AND NOT EXISTS (
        SELECT 1 FROM categoria c
        WHERE c.id = NEW.categoria_id AND c.activo
    ) THEN
        RAISE EXCEPTION 'No se puede activar un producto en una categoria inactiva';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_producto_categoria_activa
BEFORE INSERT OR UPDATE OF categoria_id, activo ON producto
FOR EACH ROW EXECUTE FUNCTION fn_producto_categoria_activa();

-- 2) No vender productos inactivos
CREATE OR REPLACE FUNCTION fn_detalle_producto_activo()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM producto p
        WHERE p.id = NEW.producto_id AND p.activo
    ) THEN
        RAISE EXCEPTION 'El producto % no esta activo', NEW.producto_id;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_detalle_producto_activo
BEFORE INSERT OR UPDATE OF producto_id ON pedido_detalle
FOR EACH ROW EXECUTE FUNCTION fn_detalle_producto_activo();

-- 3) Solo si no estaban ya en schema.sql
ALTER TABLE pedido_detalle
ADD CONSTRAINT ck_detalle_cantidad_positiva CHECK (cantidad > 0),
ADD CONSTRAINT ck_detalle_precio_no_negativo CHECK (precio_unitario >= 0);
```

2.4. Pruebas dentro de una transacción

```
BEGIN;

-- Caso valido: producto activo en categoria activa
INSERT INTO producto (nombre, precio, stock, categoria_id, activo)
VALUES ('Agua 500 ml', 900.00, 10, 1, TRUE);

-- Caso invalido: categoria 2 se da de baja y luego se intenta crear producto activo
UPDATE categoria SET activo = FALSE WHERE id = 2;
INSERT INTO producto (nombre, precio, stock, categoria_id, activo)
VALUES ('Producto invalido', 100.00, 1, 2, TRUE);
-- Esperado: ERROR por trg_producto_categoria_activa

-- Caso invalido: cantidad cero
INSERT INTO pedido_detalle (pedido_id, producto_id, cantidad, precio_unitario)
VALUES (1, 1, 0, 1000.00);
-- Esperado: viola ck_detalle_cantidad_positiva

ROLLBACK;
```

2.5. Revisión de diff y commit

```
git diff -- schema.sql
# leer cada linea antes de aplicar

git add schema.sql duia_parte1.md
git commit -m "db: garantizar productos activos y lineas de pedido validas"
```

2.6. DUIA - Parte 1

Campo	Contenido
Herramienta	OpenCode. Modelo/proveedor configurado en el entorno del alumno.
Spec o prompt utilizado	El prompt de la sección 2.2, con nombres exactos de tablas y columnas.
Qué generó	Propuesta de dos funciones PL/pgSQL, dos triggers y CHECK para pedido_detalle, además de pruebas SQL.
Qué se aceptó	La estrategia de triggers para reglas que consultan otra tabla y el uso de CHECK para reglas de una sola fila.
Qué se modificó o descartó, y por qué	Se evita duplicar CHECK ya existentes. Se conserva borrado lógico; no se agregan DELETE ni CASCADE.
Verificación realizada	INSERT válidos e inválidos dentro de BEGIN/ROLLBACK; luego repetición sobre la copia antes del COMMIT.

3. Parte 2 - Laboratorio de concurrencia con dos sesiones

Preparación común: abrir dos conexiones contra food_store_tp2. Antes de cada escenario, ambas sesiones deben confirmar la base actual. Para que los resultados sean fáciles de observar se supone producto.id = 1 con stock = 20 y categoria_id = 1.

```
-- En Sesión A y Sesión B
SELECT current_database(), current_user;

-- Preparación controlada
UPDATE producto SET stock = 20 WHERE id = 1;
```

3.1. Escenario 1 - Lectura no repetible

Objetivo: demostrar que en READ COMMITTED cada SELECT toma un snapshot nuevo y, por lo tanto, una segunda lectura dentro de la misma transacción puede observar un COMMIT ajeno.

Orden	Sesión A	Sesión B
1	BEGIN; SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	
2	SELECT stock FROM producto WHERE id=1; → 20	
3		BEGIN; UPDATE producto SET stock=18 WHERE id=1; COMMIT;
4	SELECT stock FROM producto WHERE id=1; → 18	
5	COMMIT;	

Observación: la misma consulta devuelve 20 y luego 18. Eso es una lectura no repetible. No hay lectura sucia: A solo observa el valor 18 después del COMMIT de B.

```
-- Verificación con REPEATABLE READ
-- Sesión A
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT stock FROM producto WHERE id = 1; -- 20

-- Sesión B
BEGIN;
UPDATE producto SET stock = 18 WHERE id = 1;
COMMIT;

-- Sesión A
SELECT stock FROM producto WHERE id = 1; -- sigue viendo 20
COMMIT;
```

Conclusión: REPEATABLE READ mantiene el snapshot de la transacción y evita la lectura no repetible.

3.2. Escenario 2 - Lectura fantasma

Objetivo: repetir una consulta por predicado mientras otra sesión inserta una fila que cumple el WHERE.

```
-- Sesión A
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM producto
WHERE categoria_id = 1 AND activo = TRUE; -- ejemplo: 2

-- Sesión B
BEGIN;
INSERT INTO producto (nombre, precio, stock, categoria_id, activo)
VALUES ('Fanta 1.5L', 1200.00, 15, 1, TRUE);
COMMIT;

-- Sesión A
SELECT COUNT(*) FROM producto
WHERE categoria_id = 1 AND activo = TRUE; -- ejemplo: 3
COMMIT;
```

Observación: el conjunto de filas que satisface el predicado cambia entre dos lecturas de A. En PostgreSQL READ COMMITTED esto es posible porque cada sentencia usa un snapshot propio.

```
-- Repetición en REPEATABLE READ
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM producto WHERE categoria_id=1 AND activo; -- 2
-- B inserta y hace COMMIT
SELECT COUNT(*) FROM producto WHERE categoria_id=1 AND activo; -- sigue 2
COMMIT;
```

Conclusión: en este experimento REPEATABLE READ evita que la transacción A vea la fila fantasma insertada después de su snapshot inicial.

3.3. Escenario 3 - Espera por bloqueo

Objetivo: comprobar el comportamiento de SELECT ... FOR UPDATE cuando dos sesiones intentan bloquear la misma fila.

Orden	Sesión A	Sesión B
1	BEGIN;	BEGIN;
2	SELECT * FROM producto WHERE id=1 FOR UPDATE; → obtiene lock	
3		SELECT * FROM producto WHERE id=1 FOR UPDATE; → queda esperando
4	UPDATE producto SET stock=stock-1 WHERE id=1; COMMIT;	
5		La consulta se desbloquea y devuelve la fila ya actualizada; luego COMMIT.

Conclusión: FOR UPDATE serializa la modificación de una fila concreta. La segunda sesión no obtiene una copia libre para modificar; espera a que termine la primera transacción.

3.4. Escenario adicional - Interbloqueo real

Este cuarto escenario es opcional en la guía y puede aportar evidencia adicional. Se toman dos filas en orden cruzado.

```

-- Sesión A
BEGIN;
SELECT id FROM producto WHERE id=1 FOR UPDATE;

-- Sesión B
BEGIN;
SELECT id FROM producto WHERE id=2 FOR UPDATE;

-- Sesión A: ahora pide la fila 2 y queda esperando
SELECT id FROM producto WHERE id=2 FOR UPDATE;

-- Sesión B: pide la fila 1
SELECT id FROM producto WHERE id=1 FOR UPDATE;
-- PostgreSQL detecta el ciclo y aborta una transacción:
-- ERROR: deadlock detected
-- SQLSTATE: 40P01

```

Mecanismo de prevención: adquirir locks siempre en un orden global consistente, por ejemplo, ordenar los ids ascendentemente antes de bloquear. El nivel de aislamiento por sí solo no sustituye esta disciplina de orden de bloqueo.

3.5. Resumen de verificación

Escenario	READ COMMITTED	Mecanismo que lo evita / controla
Lectura no repetible	Puede ocurrir	REPEATABLE READ mantiene snapshot estable.
Lectura fantasma	Puede ocurrir	REPEATABLE READ evita verla dentro del mismo snapshot; SERIALIZABLE agrega garantías contra anomalías de serialización.
Espera por bloqueo	La segunda sesión espera	SELECT ... FOR UPDATE; commit/rollback libera el lock.
Deadlock	Puede ocurrir con locks cruzados	Orden consistente de adquisición; PostgreSQL detecta y aborta una transacción (40P01).

3.6. DUIA - Parte 2

Campo	Contenido
Herramienta	IA generativa utilizada para reconstruir los escenarios y proponer mecanismos de aislamiento/bloqueo.
Prompt	Explicar, para cada secuencia de Sesión A/B, qué anomalía ocurre en PostgreSQL y qué aislamiento o mecanismo la evita; no asumir que la explicación es correcta sin verificar.
Qué generó	Explicaciones sobre snapshots de READ COMMITTED y REPEATABLE READ, locks de FOR UPDATE y detección de deadlocks.
Qué se aceptó	La clasificación de los tres escenarios y el mecanismo de mitigación indicado.
Qué se modificó o descartó	Se aclara que un deadlock no se “soluciona” solo elevando aislamiento: la prevención práctica es ordenar locks y manejar reintentos.
Verificación requerida	Ejecutar las dos sesiones contra la copia local y reemplazar los

Campo	Contenido
	valores de ejemplo por la salida real obtenida.

4. Parte 3 - Lectura crítica de scripts generados

4.1. Script 1

```
UPDATE funcion
SET activa = FALSE;
```

Efecto real: desactiva todas las filas de funcion porque no tiene cláusula WHERE. El comentario dice que busca dar de baja solo funciones asociadas a películas retiradas de cartel, pero el SQL no expresa esa condición.

Versión corregida, suponiendo un esquema genérico con funcion.pelicula_id y pelicula.en_cartel:

```
UPDATE funcion f
SET activa = FALSE
WHERE f.activa = TRUE
AND EXISTS (
  SELECT 1
  FROM pelicula p
  WHERE p.id = f.pelicula_id
  AND p.en_cartel = FALSE
);
```

Si el esquema real usa otra marca de retiro -por ejemplo fecha_retiro- la condición debe adaptarse a esa columna. El punto central es que el alcance del UPDATE debe quedar expresado en el WHERE antes de ejecutar.

4.2. Script 2

```
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);
```

Efecto real: pretende borrar categorías sin productos, pero NOT IN es frágil ante NULL. Si la subconsulta devuelve al menos un categoria_id NULL, la comparación puede producir UNKNOWN y el DELETE puede no borrar ninguna fila. Además, en Food Store la regla del TP1 es conservar categorías dadas de baja mediante borrado lógico.

Corrección genérica robusta con NOT EXISTS:

```
DELETE FROM categoria c
WHERE NOT EXISTS (
  SELECT 1
  FROM producto p
  WHERE p.categoria_id = c.id
);
```

Versión coherente con Food Store y su borrado lógico:

```
UPDATE categoria c
SET activo = FALSE
WHERE c.activo = TRUE
AND NOT EXISTS (
  SELECT 1
  FROM producto p
  WHERE p.categoria_id = c.id
);
```

Esta segunda variante preserva el historial y evita eliminar físicamente una categoría que podría necesitarse para auditoría o referencias históricas.

4.3. DUIA - Parte 3

Campo	Contenido
Herramienta	IA generativa como apoyo para análisis de riesgo SQL.
Prompt	Analizar los dos scripts sin ejecutarlos: determinar alcance real, riesgos y proponer una corrección mínima y una variante coherente con el borrado lógico de Food Store.
Qué generó	Detección del UPDATE sin WHERE y del problema semántico de NOT IN frente a NULL; propuestas con EXISTS/NOT EXISTS.
Qué se aceptó	La necesidad de limitar UPDATE con WHERE y sustituir NOT IN por NOT EXISTS en este caso.
Qué se modificó o descartó	Para Food Store se reemplaza DELETE físico por UPDATE activo=FALSE, respetando la decisión de diseño del TP1.
Verificación	Revisar SELECT con el mismo WHERE antes del UPDATE/DELETE y probar dentro de BEGIN ... ROLLBACK.

5. Conclusiones

Los experimentos muestran que la concurrencia no se resuelve solamente con validaciones de aplicación. PostgreSQL aporta restricciones, snapshots MVCC y bloqueos que permiten trasladar parte de la integridad al motor, pero cada mecanismo resuelve un problema distinto. READ COMMITTED favorece concurrencia a costa de permitir cambios entre sentencias; REPEATABLE READ estabiliza el snapshot; FOR UPDATE controla carreras sobre filas concretas; y los deadlocks exigen un orden de locks y una política de reintentos.

La segunda conclusión es metodológica: un script generado por IA debe considerarse una propuesta. La cadena segura es identificar el entorno, trabajar sobre copia, respaldar antes de DDL, ejecutar primero dentro de una transacción, inspeccionar el efecto y recién entonces confirmar. La salida del motor tiene prioridad sobre la explicación del modelo.

6. Checklist de entrega

- protocolo_seguridad.md adaptado al entorno y comiteado.
- Script de restricciones versionado, con git diff revisado y pruebas válidas/inválidas.
- DUIA de Parte 1 completa.
- informe_concurrencia.md con al menos tres escenarios y comandos A/B en orden.
- Salidas de concurrencia confirmadas en PostgreSQL local y no solo copiadas de esta resolución.
- DUIA de Parte 2 completa.
- ejercicio_lectura_critica.md con análisis y corrección de ambos scripts.
- DUIA de Parte 3 completa.
- Capacidad de explicar cada línea del diff y por qué se eligió cada nivel de aislamiento o lock.

7. Anexo - estructura sugerida del repositorio

```
food-store-db/  
├── AGENTS.md  
├── schema.sql  
├── protocolo_seguridad.md  
├── restricciones_tp2.sql  
├── informe_concurrencia.md  
├── ejercicio_lectura_critica.md  
├── duia_parte1.md  
├── duia_parte2.md  
├── duia_parte3.md  
└── backups/  
    └── .gitignore
```

Fin de la resolución.